

시큐어 코딩 관점에서의 상용 공유기 펌웨어 분석

곽준범, 엄홍열*

순천향대학교 (대학생)

Commercial router firmware analysis from a secure coding perspective

Jun Beom Gwak, Heung-Youl Youm*

Soonchunhyang University (University student)

요 약

본 논문은 상용화된 유무선공유기의 펌웨어를 펌웨어 추출 오픈소스인 Firmware-Mod-Kit을 이용하여 추출하고 역어셈블러 분석도구인 Ghidra를 이용하여 시큐어코딩 관점에서 취약점 분석 후 그에 따른 해결방안을 제안한다.

I. 서론

IT산업이 빠르게 성장하면서 스마트폰 어플리케이션, 가상화폐 거래소 등 IT기술을 사용하는 다양한 서비스들이 등장하였다.

하지만 이러한 서비스들이 등장하면서 관련 기업을 해킹하여 금전적인 요구를 하거나 회사의 중요정보를 유출시키는 일들이 발생하고 있다. 이러한 해킹사고는 주로 개발자들의 실수로 작성된 소프트웨어의 취약점을 공격하며, 이러한 공격을 사전에 막기 위해서는 취약점을 조기에 발견하여 이를 사전에 보완하는 것도 중요하다. 하지만, 개발단계에서 소스코드를 안전하게 작성하는 것이 매우 중요하다.

소프트웨어를 안전하게 작성하고 소프트웨어의 취약점 최소화하기 위해서는 ‘시큐어 코딩’을 이용하여 소프트웨어를 안전하게 개발해야 한다.

이에 따라 본 논문에서는 현재 상용화된 공유기의 펌웨어를 시큐어 코딩 관점에서 분석하

고 취약점을 찾아 그에 맞는 해결방안을 제안한다.

II. 시큐어 코딩의 개요

2.1 정의

행정안전부에서 정의하는 ‘소프트웨어 개발보안’의 정의는 다음과 같다.

‘SW 개발과정에서 개발자 실수, 논리적 오류 등으로 인해 SW에 내포될 수 있는 보안취약점의 원인, 즉 보안약점을 최소화하는 한편 사이버 보안위협에 대응할 수 있는 안전한 SW를 개발하기 위한 일련의 보안활동’[1]

또한 협의적 의미로는 ‘SW 개발과정 중 소스코드 구현단계에서 보안약점을 배제하기 위한 시큐어 코딩’을 말한다.[2]

2.2 국내외 현황

국내에서는 2012년 12월부터 행정안전부에 의해 시큐어 코딩에 대한 관련 법이 시행되고 있으며, 미국에서의 개인정보보호 및 시큐어코딩과 관련된 최초의 법안은 개인정보보호를 규제하는 캘리포니아 주법(SB1386)으로서 2002년

2월 12일에 캘리포니아주 상원 의원 평화의회에서 제정, 2003년 7월 1일 시행되었다.[3]

현재 행정안전부 및 KISA에서 제공하는 진단항목은 총 67개이며, 주로 JAVA, C언어를 이용하여 취약점 발생 부분 및 진단방법을 소개하고 있다.

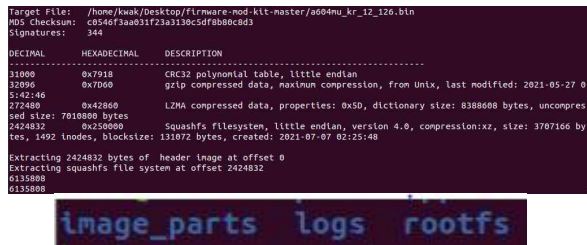
III.사용 도구

펌웨어 분석을 진행하기 위해 사용된 도구는 FMK(Firmware-Mod-Kit), Ghidra를 사용하였다.

3.1 FMK

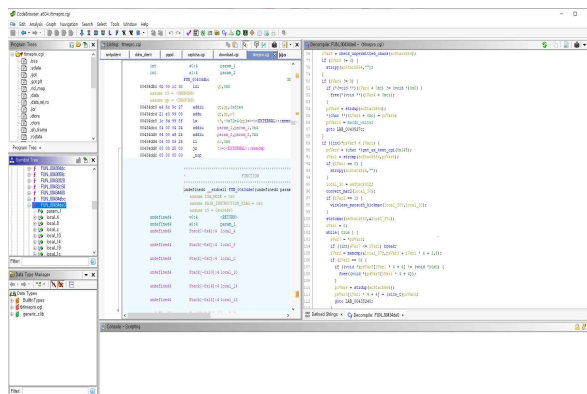
FMK는 Firmware-Mod-Kit의 약자로 Linux 기반 펌웨어 이미지를 추출하고 재구성하기 위한 스크립트 및 유틸리티 모음이다.[4]

해당 도구를 이용하여 .bin 확장자인 펌웨어 파일에서 이미지를 추출할 수 있었다.



3.2 Ghidra

Ghidra는 미국 국가 안보국에서 개발한 역어셈블러 프레임워크이다.[5] 해당 도구를 이용하여 .cgi 확장자 파일들을 Decompile하였고, 분석을 진행하였다.



IV.상용 공유기 펌웨어 분석

4.1 분석대상 공유기 정보

분석 대상이 되는 공유기의 제품명과 버전

및 해시값은 다음과 같다.[6]

iptime-A604MU, a604mu_kr_12_126.bin,
S H A 1 :
13a4df16023c42f84380240f92e73208e17e50a7

분석 대상은 rootfs 폴더 내에 있는 cgibin폴더를 분석하였으며, 해당 폴더는 .cgi확장자를 가지고 있는 파일들이 존재한다.



.cgi 파일 확장자는 CGI (Common Gateway Interface) 스크립트로 웹 서버가 동적 페이지를 생성하기 위해 실행하며 일반적으로 Perl 또는 C 프로그래밍 언어로 작성된다.[7]

cgibin 폴더 내의 cgi파일은 공유기 설정 페이지를 구성하는 파일이며, 공유기의 설정을 직접적으로 관리할 수 있는 부분이기 때문에 cgi 확장자 파일을 기준으로 분석하였다.

4.2 대상 공유기 펌웨어 취약점 식별

펌웨어 분석 결과 포맷 스트링 삽입, 부적절한 자원 해제, 메모리 버퍼 오버플로우 같은 시큐어 코딩에서 제시하는 소프트웨어 취약점이 발견되었다.

4.2.1 포맷 스트링 삽입

```
snprintf(acStack608,128,local_10);
```

포맷 스트링 삽입 취약점으로 사용될 수 있는 코드이다. 해당 코드는 snprintf함수로 acStack608의 내용을 출력할 때, 포맷스트링 인자(%s, %d, %c 등)를 사용하지 않고 그대로 출력한다.

4.2.2 부적절한 자원 해제

```
}  
free(xtable_malloc);
```

부적절한 자원 해제 취약점으로 사용될 수 있는 코드이다. 해당 코드는 free함수로 메모리를 해제한 이후, 초기화를 하지 않아 원하지 않은 코드 실행, Heap을 대상으로 하는 공격에 노출될 가능성이 존재한다.

4.2.3 메모리 오버 플로우

```
iVar1 = get_ip_value_post(param_1, "free_eip", acStack248);
if (iVar1 == 0) {
    strcpy(acStack248, acStack280);
}
```

해당 코드는 메모리 오버 플로우 취약점으로 사용될 수 있는 코드이다. strcpy함수는 NULL 문자를 만나기 전까지를 전부 복사하는데, acStack280의 값의 크기가 acStack248의 크기보다 크거나, 값이 NULL로 끝나지 않는다면 acStack248의 크기를 넘어 다른 메모리 범위를 침범할 수 있으며, 함수의 ret값을 조작할 수 있다면 공격자가 원하는 함수를 실행시키는 것이 가능하다.

4.3 예상 공격 방법

취약점이 발견되었을 때, 각 취약점들을 이용해 공격할 수 있는 예상 시나리오는 다음과 같다.

4.3.1 포맷 스트링 삽입

포맷스트링 삽입 취약점이 발생한다면 공격자가 해당 취약점을 이용하여 메모리 구조를 알아낼 수 있으며, 특정 메모리의 주소를 공격자가 값으로 변경할 수 있게 된다.

4.3.2 부적절한 자원 해제

부적절한 자원 해제 취약점이 발생한다면 공격자는 해제된 메모리를 재사용 하거나, Heap 메모리 공간에서 발생할 수 있는 UAF(Use-After-Free) 취약점을 이용하여 Heap에 저장되어 있는 값을 유추할 수 있게 된다.

4.3.3 메모리 오버 플로우

메모리 오버 플로우 취약점이 발생한다면 공격자는 값을 입력할 때 메모리 범위를 넘어선 값을 입력하거나 함수의 ret값을 원하는 함수로 조작하여 실행하는 것이 가능해진다.

V.취약점 해결 방안

발견된 소프트웨어 코드 취약점의 안전한 코드 작성을 위해서 다음과 같이 취약코드 수정 및 정탐코드 작성 방안을 제시한다.

5.1 포맷스트링 삽입 취약점 해결방안

포맷스트링 삽입 관련 부분의 해결방안은 아래의 검증코드와 같이 입력받은 값에서 포맷스트링 공격에 사용되는 포맷스트링이 있는지를 검사하거나 printf 관련함수를 이용하여 값을 출력할 때 포맷스트링 인자를 사용하는 방법이 있다.

5.1.1 입력값 포맷스트링 검사

포맷스트링 공격에서 대표적으로 사용되는 포맷스트링의 종류는 다음과 같으며. %p, %n, %hn, %hnn을 필터링 하여 메모리 변조를 막거나 메모리 주소의 출력을 방지해야만 한다.

```
if (strchr(buf, '%p') != NULL) {
    printf("format string detected!!");
    exit(1);
}
```

```
if (strchr(buf, '%n') != NULL) {
    printf("format string detected!!");
    exit(1);
}
```

5.1.2 포맷스트링 인자 사용

printf, snprintf등 printf계열의 문자열 출력 함수를 사용할 때, 적절한 포맷스트링 인자를 사용하여 출력하여야 한다.

```
snprintf(acStack608, 128, "%s", local_10)
```

5.2 부적절한 자원 해제 취약점 해결방안

부적절한 자원 해제의 해결방안은 할당받은 메모리 공간을 해제한 후, 해당 포인터에 NULL을 저장하여 예상하지 않는 코드의 실행을 막는 방법이다.

```
if(해제된 포인터 != NULL) {
    exit(1);
}
```

5.3 메모리 버퍼 오버플로우 취약점 해결방안

메모리 버퍼 오버플로우의 해결방안은 입력받은 값의 마지막에 NULL을 삽입하여 문자열의 종료를 알리거나 입력받는 버퍼의 크기보다 작은 값만 받도록 코드를 작성해야 하며, scanf 함수를 사용할때는 NULL문자까지의 길이를 고려하여 입력을 받아야 한다..

```
char buf[20] = {0};
unsigned int size = sizeof(buf);
scanf_s("%s", &buf, size);
buf[(sizeof(buf) / sizeof(char)) - 1] = '\x0';
```

VI.결론

본 논문에서는 펌웨어 추출 오픈 소스인 FMK(Firmware-Mod-Kit)을 이용하여 대상 펌웨어에서 파일이미지를 추출하고, 역어셈블리를

지원하는 오픈 소스 프로그램인 Ghidra를 이용하여 공유기 펌웨어 분석 작업을 진행하였다. 취약점 분석 결과 보안에 취약한 포맷스트링 공격, 부적절한 자원 해제, 메모리 오버 플로우 등 공격의 가능성이 있는 부분을 찾을 수 있었으며, 해당 공격의 대응 방안과 취약점을 탐지할 수 있는 코드를 제안하였다.

[참고문헌]

- [1] 전자정부 SW 개발, 운영자를 위한 소프트웨어 개발보안 가이드, 행정안전부, 한국인터넷진흥원, May, 2013
- [2] 전자정부 SW 개발, 운영자를 위한 소프트웨어 개발보안 가이드, 행정안전부, 한국인터넷진흥원, May, 2013
- [3] 최진영, 전자금융사고 예방을 위한 시큐어코딩 적용방안 연구, 금융정보화추진협의회 사무국, 한국은행 금융결제국, ISBN 978-89-97434-57-2 9332, 11월, 2012
- [4] Github - rampageX/firmware-mod-kit, <https://github.com/rampageX/firmware-mod-kit>
- [5] 기드라, 위키백과, <https://ko.wikipedia.org/wiki/%EA%B8%B0%EB%93%9C%EB%9D%BC>, 11월, 2020
- [6] ipTIME A604MU 펌웨어 버전 12.12.6, http://iptime.com/iptime/?page_id=126&dffd=1&dfsid=3&dftid=593&uid=24261&mod=document, 7월, 2021년
- [7] .cgi 파일 확장명, How2Open, <https://how2open.com/extension/cgi>